

40-Day DevOps & SRE Coding Round Roadmap

Python-first · Major-company interview prep · Built from your original 10-week roadmap

Target outcome	Be ready for typical DevOps/SRE coding rounds at large product companies in 40 days.
What changes from the old plan	Faster pacing, stronger timed practice, Python fluency, infra-flavored coding, and mock-round pressure every week.

Who this is for: An infra engineer who already understands real systems, but wants fast, repeatable performance in coding rounds. This is not a beginner Go/Python guide and not a 300-LeetCode grind.

What this roadmap optimizes for

- Passing the coding screen and the first on-site coding round at major DevOps/SRE-heavy companies.
- Becoming fluent in the 8–10 patterns that appear most often: graphs, heaps, sliding window, intervals, binary search, trees, tries, cache design, and a light layer of DP.
- Speaking like an infra engineer while solving: failure awareness, trade-offs, and production analogies.
- Building enough Python speed that your implementation stops being the bottleneck.

Ground rules

- Every solve is verbal. Explain the brute force first, then the optimized plan, then code.
- Use Python 3 throughout. Lean on deque, heapq, defaultdict, Counter, bisect, and OrderedDict without guilt.
- Do one timed problem almost every day. Timed composure matters more than endless untimed grinding.
- Keep a failure log with three lines: what confused you, what pattern you missed, what you will say next time.
- Bridge every problem to infra after solving it: rate limiting, scheduling, DAGs, caches, dependency graphs, or stream processing.

Default daily cadence (2 to 2.5 hours)

- 35–45 min: one timed problem, no hints.

- 35–45 min: full cleanup solve or second related problem untimed.
- 20 min: pattern note or one excellent solution review.
- 15 min: write complexity and failure note; connect it to SRE/DevOps context.
- Optional 20 min: Python drill or mini-design prompt if you have the energy.

The interview protocol you should memorize

- Minute 0–1: restate the problem and ask one clarifying question.
- Minute 1–4: state a brute-force approach and why it is too slow.
- Minute 4–5: commit to the optimized pattern and justify it.
- Minute 5–22: code while narrating decisions, edge cases, and invariants.
- Minute 22–25: test on one normal example and one nasty edge case, then state time and space complexity cleanly.

What actually shows up in DevOps/SRE coding rounds

Pattern	Why it matters	Representative problems
Graphs / BFS / DFS / Union-Find	Dependency graphs, reachability, topology, scheduler constraints, cycle detection.	Number of Islands, Course Schedule, Redundant Connection, Network Delay Time
Heaps / Priority Queues	Scheduling, top-K hot spots, merging streams, running medians.	Kth Largest, Task Scheduler, Merge K Sorted Lists, Median from Data Stream
Sliding Window / Deque	Rate limiting, rolling metrics, anomaly windows, log scans.	Longest Substring Without Repeat, Minimum Window, Sliding Window Maximum, custom rate limiter
Intervals / Two Pointers / Binary Search	Scheduling windows, capacity thresholds, sorted streams, SLO search-on-answer.	Merge Intervals, Insert Interval, Koko, Ship Packages
Trees / Tries / Cache Design	Prefix lookup, config trees, DNS-like hierarchies, local client caching.	Level Order, Validate BST, Implement Trie, LRU Cache

Phase 1 • Days 1–10 • Core patterns + Python speed

Day	Focus	Core work	Deliverable / check
1	Setup + graph baseline	Write your solve protocol. Solve Number of Islands timed. Then re-solve cleanly with BFS and DFS.	1-page failure log starts today.
2	Cycle detection / topo sort	Course Schedule. Explain why DFS color-marking and Kahn both work.	Say complexity without looking at notes.
3	Weighted graphs	Network Delay Time. Implement Dijkstra with heapq from scratch.	Explain adjacency list + heap trade-off.
4	Union-Find	Redundant Connection plus one fast DSU template rewrite.	Template fits in 12–15 lines from memory.
5	Heap fundamentals	Kth Largest + K Closest Points. Practice min-heap vs simulated max-heap.	No heapq confusion under timer.
6	Merge and scheduling heaps	Merge K Sorted Lists or arrays, then Task Scheduler.	Explain why heap beats repeated sorting.
7	Timed mock 1	1 graph + 1 heap problem back to back. 55 total minutes.	Review recording and note dead air.
8	Sliding window basics	Longest Substring Without Repeat, then Max Average Subarray.	Say what makes the window invalid.
9	Harder windows	Minimum Window Substring or Sliding Window Maximum with deque.	Articulate invariant before coding.
10	Python speed drill	Rewrite templates for BFS, DFS, heap, window, binary search from memory.	Single cheatsheet page created.

Phase 2 • Days 11–20 • Interview-grade pattern control

Day	Focus	Core work	Deliverable / check
11	Two pointers	Two Sum II + 3Sum. Practice sorted-array reasoning out loud.	No apology for $O(n^2)$ when justified.
12	Intervals	Merge Intervals + Insert Interval.	Say the sort-first argument crisply.
13	Binary search template	Binary Search + Search in Rotated Sorted Array.	Template has no off-by-one bugs.
14	Search on answer	Koko Eating Bananas or Ship Packages Within D Days.	Explain feasible(mid) like a grown-up.
15	Trees	Level Order + Max Depth + Diameter.	Can switch between BFS and DFS naturally.
16	BST / Trie	Validate BST + Implement Trie.	Tie trie to etcd/Consul prefix lookups.
17	Cache design	LRU Cache in Python using OrderedDict or manual DLL+hashmap if needed.	Explain $O(1)$ operations clearly.
18	Rate limiter design	Implement a sliding-window rate limiter; discuss token bucket vs leaky bucket.	Can bridge code to infra in 60 seconds.
19	Timed mock 2	1 sliding-window/interval + 1 tree/binary-search problem.	Track bugs caused by rushing.
20	Weak spot repair	Redo your worst 3 problems from Days 1–19 without notes.	Failure patterns summarized in one page.

Phase 3 • Days 21–30 • Company-style coding pressure

Day	Focus	Core work	Deliverable / check
21	Top-K + frequency	Top K Frequent Elements and discuss heap vs bucket trade-off.	Know when $O(n)$ bucket is valid.
22	Matrix / grid BFS	01 Matrix, Walls and Gates, or Rotting Oranges.	Explain boundary handling cleanly.
23	Graph hard-ish	Word Ladder or Cheapest Flights Within K Stops.	Narrate state representation explicitly.
24	Median / dual heap	Find Median from Data Stream or equivalent running-metric problem.	Explain rebalance invariants.
25	Stack / monotonic flavor	Min Stack plus one monotonic-stack or deque problem.	Add to your compact template sheet.
26	Lite DP only	House Robber + Coin Change.	Memoization vs bottom-up distinction is automatic.
27	Infra-flavored custom problem	Implement merge of sorted log streams or health-check window detector.	One polished GitHub gist or local repo file.
28	Mock 3	One full 45-minute coding interview with narration, then 20-minute debrief.	Focus on clarity, not just correctness.
29	Python cleanup day	Practice collections, sorting with keys, bisect, and writing tests fast.	Your language friction should feel low.
30	Rescue day	Redo hardest graph, hardest heap, hardest window.	All three solved with calm narration.

Phase 4 • Days 31–40 • Final round readiness

Day	Focus	Core work	Deliverable / check
31	Mock 4	Graph + heap under strict timer.	No long silent gaps.
32	Mock 5	Binary search + intervals under strict timer.	Complexity stated before prompting.
33	Mock 6	Tree/trie/cache design combo.	Use production analogies naturally.
34	System-design bridge	For each core pattern, write one DevOps/SRE system analogy.	1-page bridge sheet finished.
35	Design-lite coding	LRU cache + rate limiter back to back in 60 minutes.	This is the classic SRE flex round.
36	Randomized cold start	Take two unseen medium problems from your target patterns.	Measure first-attempt composure.
37	Failure-note review	Read every failure note. Create top 5 recurring interview mistakes.	Each mistake gets a fix rule.
38	Confidence set	Solve your 5 best problems in one sitting.	Rebuild rhythm and confidence.
39	Final full mock	90-minute simulation: 2 coding problems plus 10-minute wrap-up.	Interview-ready scorecard completed.
40	Light review only	Read templates, complexity rules, and analogies. No heavy grinding.	Go into interviews sharp, not cooked.

Problem bank for the 40 days

Must master (non-negotiable)

- Number of Islands
- Course Schedule
- Network Delay Time
- Redundant Connection
- Kth Largest Element
- Task Scheduler
- Longest Substring Without Repeat
- Sliding Window Maximum
- Merge Intervals
- Insert Interval
- Binary Search
- Koko Eating Bananas
- Ship Packages Within D Days
- Binary Tree Level Order
- Validate BST
- Implement Trie
- LRU Cache

High value second tier

- K Closest Points
- Merge K Sorted Lists
- Minimum Window Substring
- 3Sum
- 01 Matrix
- Word Ladder
- Top K Frequent Elements
- Find Median from Data Stream
- House Robber
- Coin Change
- Cheapest Flights Within K Stops
- Min Stack

Custom infra implementations

- Sliding-window rate limiter
- Merge sorted log streams
- Health-check window detector
- Simple in-memory LRU
- Top-K noisy service tracker

Python fluency checklist for coding rounds

- Use deque for BFS, not list.pop(0).
- Use heapq comfortably, including tuple ordering and negative values for max-heap behavior.
- Use defaultdict(list), Counter, and OrderedDict without pausing to remember syntax.
- Know sorted(..., key=...), list.sort(...), enumerate, zip, and set/dict comprehensions cold.
- Keep one binary-search template, one DFS/BFS template, one union-find template, and one sliding-window template in memory.

What interviewers at bigger companies usually care about

- You choose the right pattern without thrashing.
- You communicate trade-offs and edge cases clearly.
- Your Python is clean and idiomatic, not clever for the sake of it.
- You can connect a coding problem to production systems naturally.
- You recover gracefully when stuck instead of freezing.

What to de-prioritize

- Heavy 2D dynamic programming marathons.
- Exotic graph theory that rarely appears in infra-leaning interviews.
- Language-agnostic pseudo-code drills that never become executable speed.
- Huge problem counts with zero review.

Weekly scorecard

Week/Phase	Timed problems done	Mocks completed	Weak patterns	Next fixes
Phase 1				
Phase 2				
Phase 3				
Phase 4				

Final note: The original 10-week roadmap was strong for pattern coverage, but this rewrite compresses it into 40 days by making mocks frequent, removing low-yield filler, and forcing Python execution speed. That is the right trade if your goal is interviews soon, not leisurely algorithm study.